Linear regression: predicting numbers

🎯 Today's goal

By the end of this hour you will:

- understand a straight line `y = a·x + b` as a **prediction rule**
- see what **residuals** (the misses) are and why the "best" line makes them small
- fit a line with NumPy's `polyfit` and with scikit-learn's `LinearRegression`
- measure the errors with **MAE**, **RMSE** and **R^2**
- use **several features** at once, and know when *not* to trust a prediction

🤔 Why this matters

Yesterday the model predicted a *category* (a flower species). That's **classification**. Today it predicts a *number*: a price, a temperature, a number of visitors. That's **regression**.

The simplest regression model is a straight line. If you've ever thought "a bigger flat costs more, about this much per square metre", you already had a linear model in your head. **Linear regression** finds the line that fits the data best, and it's still one of the most used tools in data science because it's fast and easy to explain.

🏠 The data: apartment sizes and prices

We make up 40 apartments. Each has a size in square metres (m^2) and a price in thousands of made-up dollars. We built the prices as "about 2.5 per m^2, plus 40, plus some random noise", but let's pretend we don't know that and see whether the computer can find it.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
# 40 sizes between 30 and 120 m²
size = rng.uniform(30, 120, 40).round()
# made-up prices in thousands: 2.5 per m², plus 40, plus noise
price = (2.5 * size + 40 + rng.normal(0, 20, 40)).round()

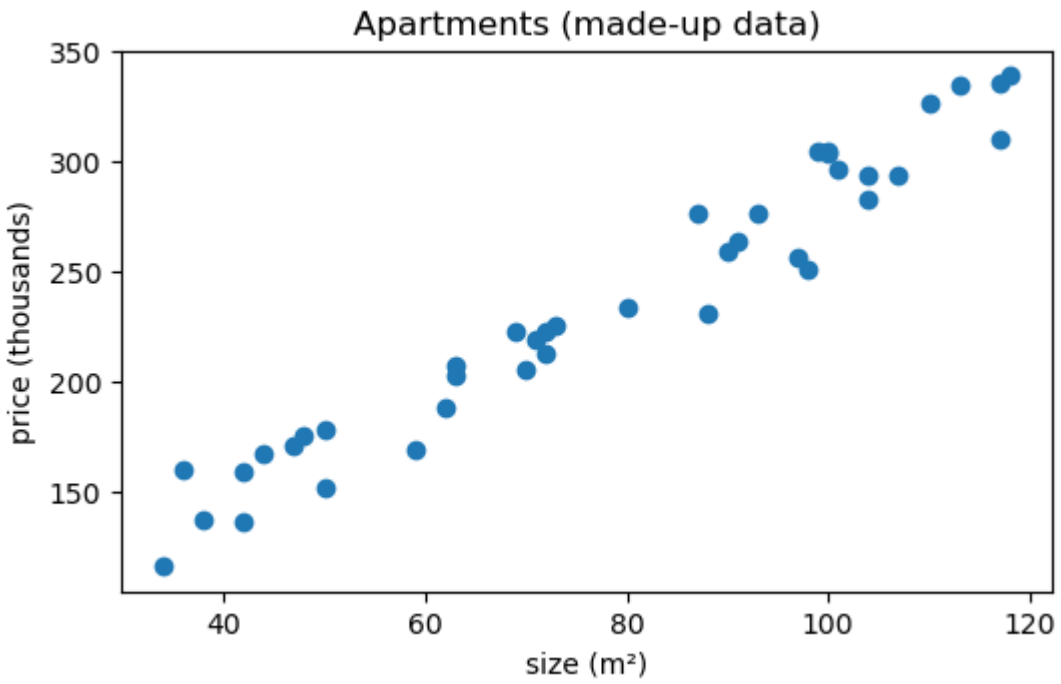
apartments = pd.DataFrame({"size_m2": size, "price": price})
apartments.head()
```

	size_m2	price
0	100.0	305.0
1	69.0	223.0
2	107.0	294.0
3	93.0	277.0
4	38.0	137.0

Always look first. A scatter plot (Day 13) shows whether a straight line makes sense at all:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 3.5))
plt.scatter(apartments["size_m2"], apartments["price"])
plt.title("Apartments (made-up data)")
plt.xlabel("size (m²)")
plt.ylabel("price (thousands)")
plt.show()
```



The dots climb from left to right in a rough band. That's a good sign for a straight line.

📏 A line is a prediction rule

A straight line is written as:

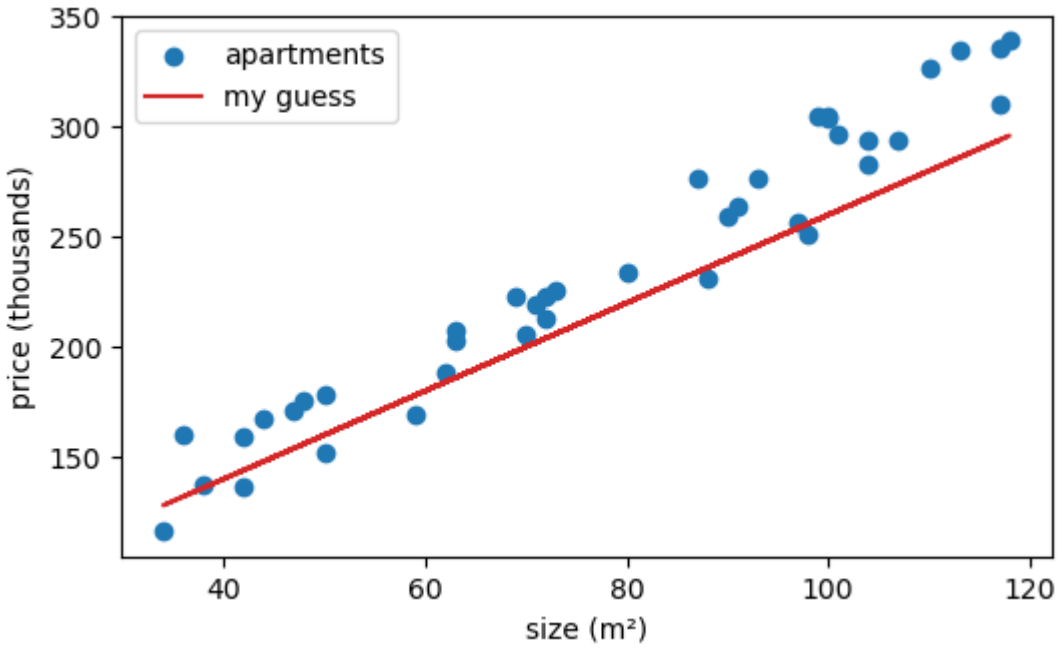
`price = a × size + b`

- `a` is the **slope**: how much the price goes up for each extra m².
- `b` is the **intercept**: where the line starts when the size is 0.

Pick any `a` and `b` and you have a model! Let's guess by eye: 2 per m², starting at 60:

```
a = 2
b = 60
guess = a * apartments["size_m2"] + b    # a predicted price for each one

plt.figure(figsize=(6, 3.5))
plt.scatter(apartments["size_m2"], apartments["price"], label="apartments")
plt.plot(apartments["size_m2"], guess, color="tab:red", label="my guess")
plt.xlabel("size (m²)")
plt.ylabel("price (thousands)")
plt.legend()
plt.show()
```



Not bad, but is it the *best* line? To answer that, look at the **residuals**: for each apartment, the real price minus the predicted price. A residual is how much the line missed by. Positive means the line guessed too low, negative too high.

```
residuals = apartments["price"] - guess
print("First five misses:", residuals.head().tolist())
print("Average miss (sign ignored):", residuals.abs().mean().round(1))
```

```
First five misses: [45.0, 25.0, 20.0, 31.0, 1.0]
Average miss (sign ignored): 21.6
```

`.abs()` removes the minus signs, so misses in both directions count. Try other values for `a` and `b` in the box above and run both boxes again. Can you get the average miss lower?

🎯 Let the computer find the best line

Trying values by hand is slow. The **best-fit line** is the one that makes the residuals as small as possible (to be exact, it makes the sum of the *squared* residuals as small as possible, which is why it's also called "least squares"). NumPy finds it in one line. The `1` means "a straight line" (a curve would need a bigger number):

```
best_a, best_b = np.polyfit(apartments["size_m2"], apartments["price"], 1)
print(f"Best slope a:     {best_a:.2f} per m²")
print(f"Best intercept b: {best_b:.2f}")
```

```
Best slope a:     2.37 per m²
Best intercept b: 50.24
```

Fairly close to the 2.5 and 40 we used to make the data. It isn't exact, because the random noise pushes things around a little. With real data nobody tells you the true numbers, so the fitted line is your best estimate.

🤖 The same with scikit-learn

`polyfit` is handy for one feature. For real projects we use scikit-learn's `LinearRegression`, which follows yesterday's steps (create → fit → predict → score) and works with many features. And, like yesterday, we keep a test set aside so we can judge the model fairly:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X = apartments[["size_m2"]]      # features: a table, so double brackets
y = apartments["price"]          # label: the number we want to predict

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(f"Slope a:     {model.coef_[0]:.2f}")    # coef_: one slope per feature
print(f"Intercept b: {model.intercept_:.2f}")
```

```
Slope a:     2.29
Intercept b: 57.79
```

The numbers differ a little from `polyfit`'s, because this model learned from only the 30 training apartments. `coef_` (short for *coefficients*) is a list with one slope per feature; we have one feature,

so we take `coef_[0]`. The trailing underscore is scikit-learn's way of saying "learned during fit".

Forget the double brackets and scikit-learn complains, because `X` must be a table:

💡 This example has a mistake on purpose. Run it and read the error.

```
# single brackets: just one column, not a table
LinearRegression().fit(apartments["size_m2"], y)
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 2, in <module>
>     LinearRegression().fit(apartments["size_m2"], y)
>     ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^
> ValueError: Expected a 2-dimensional container but got <class 'pandas.Series'>
> instead. Pass a DataFrame containing a single row (i.e. single sample) or a
> single column (i.e. single feature) instead.
> ```

"Expected a 2-dimensional container": two dimensions means rows *and* columns. `apartments[["size_m2"]]` fixes it.

🔮 Predicting new prices

Now the fun part. What should a 50, 80 or 100 m² apartment cost? New examples go in a DataFrame with the same column name as `X`:

```
new_flats = pd.DataFrame({"size_m2": [50, 80, 100]})
new_prices = model.predict(new_flats)

for s, p in zip(new_flats["size_m2"], new_prices):
    print(f"{s} m² → about {p:.0f} thousand")
```

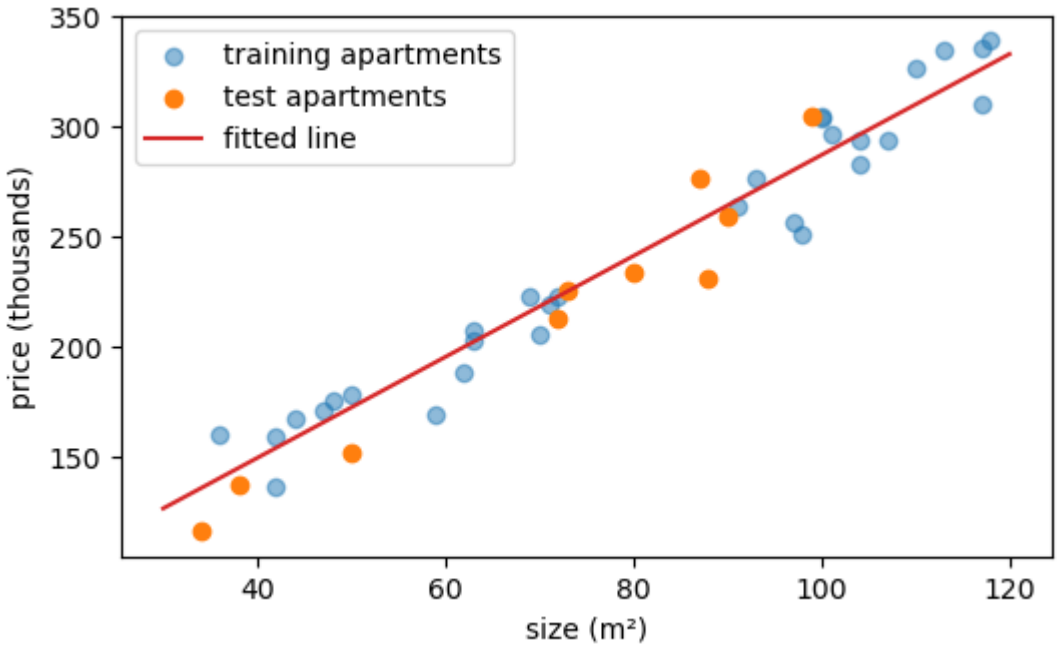
```
50 m² → about 172 thousand
80 m² → about 241 thousand
100 m² → about 287 thousand
```

`zip` walks through two lists side by side, giving you one item from each at every step.

Let's draw the fitted line through the test apartments the model has never seen:

```
line_x = pd.DataFrame({"size_m2": [30, 120]})    # 2 points make a line
line_y = model.predict(line_x)

plt.figure(figsize=(6, 3.5))
plt.scatter(X_train["size_m2"], y_train, label="training apartments", alpha=0.5)
plt.scatter(X_test["size_m2"], y_test, label="test apartments")
plt.plot(line_x["size_m2"], line_y, color="tab:red", label="fitted line")
plt.xlabel("size (m²)")
plt.ylabel("price (thousands)")
plt.legend()
plt.show()
```



📐 How wrong are the predictions?

For numbers, "accuracy" (right or wrong) makes no sense: a prediction of 230 for a real price of 231 isn't "wrong". Instead we measure *how far off* the predictions are, on the **test set**:

Measure	What it means	Good value
MAE (mean absolute error)	the average miss, ignoring the sign, in the same units as the price	small
RMSE (root mean squared error)	square the misses, average them, take the square root. Like MAE, but big misses count extra	small
R^2 ("R squared")	how much of the ups and downs in the prices the model explains. 1 = perfect, 0 = no better than always guessing the average price	close to 1

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

test_predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, test_predictions)
rmse = np.sqrt(mean_squared_error(y_test, test_predictions))
r2 = r2_score(y_test, test_predictions)

print(f"MAE:  {mae:.1f} thousand")
print(f"RMSE: {rmse:.1f} thousand")
print(f"R²:   {r2:.2f}")
```

```
MAE:  14.0 thousand
RMSE: 16.3 thousand
R²:   0.92
```

In words: on apartments it has never seen, the model is off by about 14 thousand on average, and it explains about 92% of the differences in price. `model.score(X_test, y_test)` gives the same R^2, just like it gave accuracy for yesterday's classifier. MAE is easy to see by hand, too: it's the mean of the absolute residuals, `(y_test - test_predictions).abs().mean()`.

🚧 Don't trust the line too far

The model only saw apartments between 30 and 120 m^2. Ask it about sizes far outside that range (**extrapolation**) and it happily answers, but nothing says the straight line continues out there:

```
odd_flats = pd.DataFrame({"size_m2": [0, 5, 1000]})
for s, p in zip(odd_flats["size_m2"], model.predict(odd_flats)):
    print(f"{s} m² → {p:.0f} thousand")
```

```
0 m² → 58 thousand
5 m² → 69 thousand
1000 m² → 2352 thousand
```

An apartment of 0 m² that costs 58 thousand? A 1000 m² palace whose price just follows the same line, although no apartment that big was in the data? A model only knows the world it was trained on. Predict *inside* the range of your data.

🧩 Several features at once

Real prices depend on more than size (location, age, floor…). Linear regression can use many features: it simply learns one slope for each, and adds them all up: `prediction = a1 × feature1 + a2 × feature2 + … + b`.

scikit-learn includes a real dataset to try this: `load_diabetes`. It has 442 patients, 10 features (age, sex, body-mass index `bmi`, blood pressure `bp`, and six blood measurements `s1` – `s6`) and a label that measures how much the illness progressed one year later (a bigger number = worse).

Teaching example only: we use this medical dataset to practise regression. Nothing here is medical advice. Also, the features have already been rescaled to small numbers around 0, so an "age" of 0.038 doesn't mean a real age.

```
from sklearn.datasets import load_diabetes

diabetes = load_diabetes(as_frame=True)
X_d = diabetes.data
y_d = diabetes.target

print("Features:", X_d.columns.tolist())
print("Label: from", y_d.min(), "to", y_d.max())
X_d[["age", "sex", "bmi", "bp", "s1", "s5"]].head(3).round(3)   # 6 of 10
```

```
Features: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
Label: from 25.0 to 346.0
```

	age	sex	bmi	bp	s1	s5
0	0.038	0.051	0.062	0.022	-0.044	0.020
1	-0.002	-0.045	-0.051	-0.026	-0.008	-0.068
2	0.085	0.051	0.044	-0.006	-0.046	0.003

The steps are exactly the same as with one feature. Only `X` is wider:

```
Xd_train, Xd_test, yd_train, yd_test = train_test_split(
    X_d, y_d, test_size=0.2, random_state=42)

big_model = LinearRegression()
big_model.fit(Xd_train, yd_train)

print("Slopes learned:", len(big_model.coef_))
print(f"R² on the test set:  {big_model.score(Xd_test, yd_test):.2f}")
big_mae = mean_absolute_error(yd_test, big_model.predict(Xd_test))
print(f"MAE on the test set: {big_mae:.1f}")
```

```
Slopes learned: 10
R² on the test set:  0.45
MAE on the test set: 42.8
```

An R^2 of about 0.45: much better than guessing the average, but far from perfect. That's normal for real data about people: many things that matter aren't in the table. A straight-line model can only be as good as the features you give it.

⚠️ Common mistakes

- Passing one column with single brackets as `X` → "Expected a 2-dimensional container". Use `df[["col"]]`.
- Judging the line on the training data instead of the test set.
- Using accuracy for numbers. Regression uses MAE, RMSE and R^2.
- Trusting predictions far outside the range of the data (extrapolation).
- Forgetting that `coef_` is a list: with one feature, the slope is `coef_[0]`.

✅ Recap

- A line `y = a·x + b` is a model: slope `a`, intercept `b`.
- Residual = real value – predicted value. The best-fit line makes the squared residuals as small as possible.
- `np.polyfit(x, y, 1)` returns `a, b`.
- `LinearRegression()` → `fit` → `coef_`, `intercept_`, `predict`, `score` (R^2).
- MAE = average miss; RMSE punishes big misses; R^2 = share of the variation explained.
- More features: same code, a wider `X`, one slope per feature.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Study hours and test scores

Made-up data: hours studied and the test score of 8 students. Use `np.polyfit` to find the best straight line and store its slope in `slope` and its intercept in `intercept`.

```
import numpy as np

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8])
scores = np.array([52, 55, 61, 64, 70, 71, 78, 83])

# slope, intercept = ...
```

Hints:

1. `np.polyfit(x, y, 1)`: first the x values (hours), then the y values (scores), then `1` for a straight line.
2. It returns two numbers, slope first: `slope, intercept = np.polyfit(hours, scores, 1)`

2. Ice cream and temperature

A made-up ice-cream stand recorded the temperature and the number of ice creams sold. Fit a `LinearRegression` called `model` on **all** the data (temperature → sales), then store the slope in `slope` and the intercept in `intercept`.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

stand = pd.DataFrame({
    "temperature": [18, 21, 24, 26, 29, 31, 33, 35],
    "sales": [40, 52, 63, 70, 85, 91, 99, 108],
})

# model = ...
# slope = ...
# intercept = ...
```

Hints:

1. Features need double brackets: `stand[["temperature"]]`; the label is `stand["sales"]`.
2. `model = LinearRegression()`, then `model.fit(X, y)`.
3. `slope = model.coef_[0]` and `intercept = model.intercept_`

3. Predict tomorrow's sales

The model from exercise 2 is ready in the starter. The weather forecast for the next three days says 20, 27 and 32 degrees. Store the model's predicted sales for those days in `predictions`.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

stand = pd.DataFrame({
    "temperature": [18, 21, 24, 26, 29, 31, 33, 35],
    "sales": [40, 52, 63, 70, 85, 91, 99, 108],
})
model = LinearRegression()
model.fit(stand[["temperature"]], stand["sales"])

# predictions = ...
```

Hints:

1. Put the new temperatures in a DataFrame with the same column name: `pd.DataFrame({"temperature": [20, 27, 32]})`
2. `predictions = model.predict(forecast)`

4. How far off?

A made-up weather model predicted 6 daily temperatures. Calculate the **MAE** in `mae` and the **RMSE** in `rmse`. You can use scikit-learn or do it by hand with NumPy.

```
import numpy as np

real = np.array([21.0, 23.5, 19.0, 25.0, 27.5, 22.0])
predicted = np.array([20.0, 24.5, 21.0, 24.0, 25.5, 22.5])

# mae = ...
# rmse = ...
```

Hints:

1. The misses are `real - predicted`. MAE is the mean of their absolute values: `np.abs(misses).mean()`.
2. RMSE: square the misses (`** 2`), take the mean, then `np.sqrt`.
3. Or with scikit-learn: `mean_absolute_error(real, predicted)` and `np.sqrt(mean_squared_error(real, predicted))` from `sklearn.metrics`.

5. Challenge: do more features help?

With the diabetes data and the split in the starter, train two linear regressions: one that uses **only** `bmi`, and one that uses the three features `bmi`, `bp` and `s5`. Store their R^2 on the **test** set in `r2_one` and `r2_three`.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

diabetes = load_diabetes(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data, diabetes.target, test_size=0.2, random_state=42)

# r2_one = ...
# r2_three = ...
```

Hints:

1. Pick columns from the training and the test tables: `X_train[["bmi"]]` and `X_test[["bmi"]]`.
2. For three features: `cols = ["bmi", "bp", "s5"]`, then `X_train[cols]` and `X_test[cols]`.
3. `r2_one = LinearRegression().fit(X_train[["bmi"]], y_train).score(X_test[["bmi"]], y_test)`, and the same for the three columns.

➡️ Tomorrow

Classification: logistic regression and KNN. Back to predicting categories, but this time the model will also tell you *how sure* it is (a probability). You'll meet logistic regression, look inside k-nearest neighbours, learn why features must be put on the same scale, and compare models on a breast-cancer dataset (as a teaching example).

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Study hours and test scores

```
import numpy as np

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8])
scores = np.array([52, 55, 61, 64, 70, 71, 78, 83])

slope, intercept = np.polyfit(hours, scores, 1)
print(f"Each extra hour adds about {slope:.1f} points.")
```

2. Ice cream and temperature

```
import pandas as pd
from sklearn.linear_model import LinearRegression

stand = pd.DataFrame({
    "temperature": [18, 21, 24, 26, 29, 31, 33, 35],
    "sales": [40, 52, 63, 70, 85, 91, 99, 108],
})

model = LinearRegression()
model.fit(stand[["temperature"]], stand["sales"])
slope = model.coef_[0]
intercept = model.intercept_
print(f"Each extra degree sells about {slope:.1f} more ice creams.")
```

3. Predict tomorrow's sales

```
import pandas as pd
from sklearn.linear_model import LinearRegression

stand = pd.DataFrame({
    "temperature": [18, 21, 24, 26, 29, 31, 33, 35],
    "sales": [40, 52, 63, 70, 85, 91, 99, 108],
})
model = LinearRegression()
model.fit(stand[["temperature"]], stand["sales"])

forecast = pd.DataFrame({"temperature": [20, 27, 32]})
predictions = model.predict(forecast)
print(predictions.round(0))
```

4. How far off?

```
import numpy as np

real = np.array([21.0, 23.5, 19.0, 25.0, 27.5, 22.0])
predicted = np.array([20.0, 24.5, 21.0, 24.0, 25.5, 22.5])

misses = real - predicted
mae = np.abs(misses).mean()
rmse = np.sqrt((misses ** 2).mean())
print(f"MAE: {mae:.2f}  RMSE: {rmse:.2f}")
```

5. Challenge: do more features help?

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

diabetes = load_diabetes(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data, diabetes.target, test_size=0.2, random_state=42)

one = LinearRegression()
one.fit(X_train[["bmi"]], y_train)
r2_one = one.score(X_test[["bmi"]], y_test)

cols = ["bmi", "bp", "s5"]
three = LinearRegression()
three.fit(X_train[cols], y_train)
r2_three = three.score(X_test[cols], y_test)

print(f"bmi only: R² = {r2_one:.2f}")
print(f"bmi + bp + s5: R² = {r2_three:.2f}")
```